

FM SCA Subcarrier Receiver

(`fm_sca_rx.py` & `fm_sca_rx_gui.py`)

Overview

This documentation covers two Python/GNU Radio scripts designed to receive, visualize, and demodulate FM Subsidiary Communications Authorization (SCA) subcarriers using an RTL-SDR receiver (such as the RTL-SDR Blog V4).

SCA subcarriers (typically centered at 67 kHz or 92 kHz) sit high in the FM baseband multiplex (MPX) spectrum, riding above the main mono audio, 19 kHz stereo pilot, and 57 kHz RDS data.

- **`fm_sca_rx.py`:** A CLI-only receiver that demodulates SCA subcarrier audio directly to the default system audio output.
- **`fm_sca_rx_gui.py`:** An interactive GUI receiver featuring an FFT plot focused specifically on the high-frequency subcarrier region (45 kHz to 105 kHz) to assist in identifying active subcarrier sidebands and carrier frequencies.

Dependencies & Requirements

- **Python:** 3.10+
- **GNU Radio:** 3.10.x (utilizing updated `fft.window` and `IIR filter` syntax)
- **Python Libraries:** PyQt5, sip, argparse
- **Hardware Drivers:** `gr-osmosdr`
- **Supported Hardware:** RTL-SDR Blog V4 (or any OsmosDR-compatible hardware)

Command-Line Arguments

Both scripts accept identical command-line flags:

Flag	Long Flag	Description	Default
-f	--freq	Target FM station frequency in MHz	88.5
-s	--sca	Target SCA subcarrier frequency in kHz	67.0
-g	--gain	RTL-SDR RF gain in dB	40.0

Usage Examples

Command-Line Receiver (fm_sca_rx.py)

Run the CLI receiver on 96.1 MHz tuned to a 67 kHz subcarrier with 40 dB gain:

```
python fm_sca_rx.py -f 96.1 -s 67 -g 40
```

Run on 100.3 MHz listening to a 92 kHz subcarrier with 35 dB gain:

```
python fm_sca_rx.py -f 100.3 -s 92 -g 35
```

GUI Subcarrier Monitor (fm_sca_rx_gui.py)

Launch the graphical subcarrier monitor focused on 45 kHz–105 kHz:

```
python fm_sca_rx_gui.py -f 100.3 -s 67 -g 32
```

NOTE: Make the scripts executable as follows and be sure to include them in your search path.

```
% chmod +x /Applications/radioconda/bin/fm_sca_rx.py
% chmod +x /Applications/radioconda/bin/fm_sca_rx_gui.py
% echo 'export PATH="/Applications/radioconda/bin:$PATH"' >> ~/.zshrc
% source ~/.zshrc
```

These scripts are now fully executable from anywhere as:

```
% fm_sca_rx_gui.py -f 100.3 -s 67 -g 20
gr-osmosdr 0.2.0.0 (0.2.0) gnuradio 3.10.12.0
built-in source types: file rtl rtl_tcp uhd miri hackrf bladerf rfspace airspy airspyhf soapy redpitaya
Using device #0 RTLSDRBlog Blog V4 SN: 00000001
Found Rafael Micro R828D tuner
RTL-SDR Blog V4 Detected
```

--- SCA Subcarrier Monitor Running ---

Station Frequency: 100.3 MHz

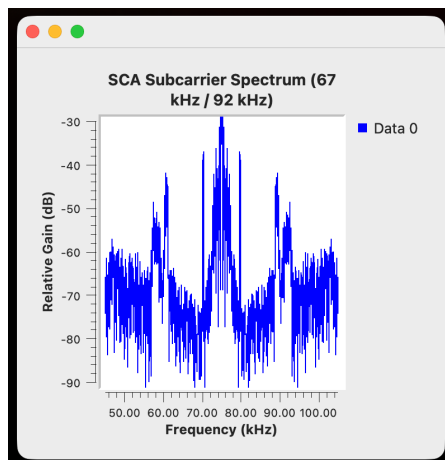
SCA Subcarrier: 67.0 kHz

RF Gain: 20.0 dB

Displaying region: 45 kHz to 105 kHz

Close GUI window to exit.

aUaUaUaUaUaUaUaUaUaUaUaUaUaUaUaUaU



-- Or --

```
% fm_sca_rx.py -f 100.3 -s 67 -g 20
gr-osmosdr 0.2.0.0 (0.2.0) gnuradio 3.10.12.0
built-in source types: file rtl rtl_tcp uhd miri hackrf bladerf rfspc airspy airspyhf soapy redpitaya
Using device #0 RTLSDRBlog Blog V4 SN: 00000001
Found Rafael Micro R828D tuner
RTL-SDR Blog V4 Detected
```

```

--- SCA Receiver Running ---
Station Frequency: 100.3 MHz
SCA Subcarrier:    67.0 kHz
RF Gain:          20.0 dB

```

Press Ctrl+C or Enter to stop.

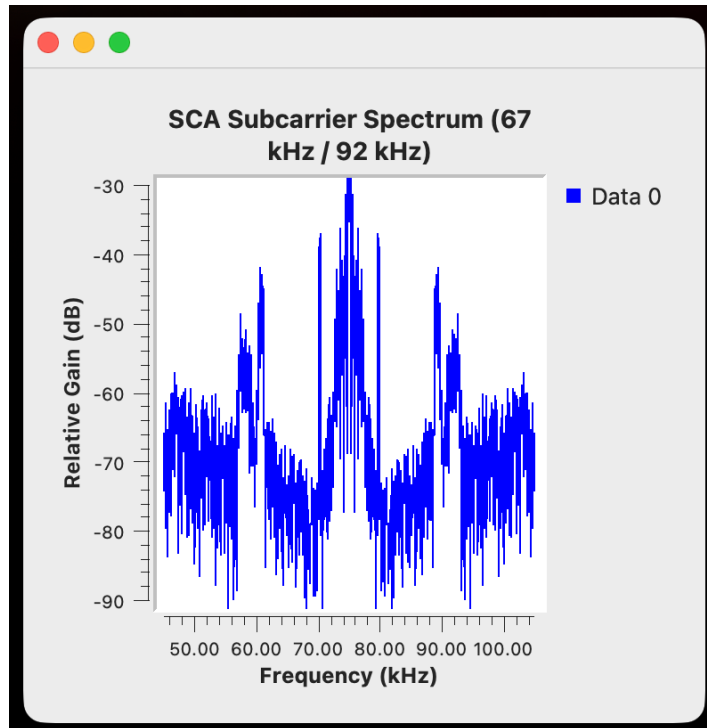
aUaUaUaUaUaUaUaUaUaUaUaUaUaUaUaUaU

DSP Architecture & Flowgraph Breakdown

1. **RTL-SDR Source** (`osmosdr.source`): Captures RF spectrum at 2.4 MSps centered on `-f/--freq`.
2. **Wideband FM Translating Filter** (`filter.freq_xlating_fir_filter_ccc`): Decimates 2.4 MSps down to 240 kHz (Decimation = 10) using an 80 kHz low-pass FIR tap filter to isolate the target station MPX.
3. **Wideband FM Quadrature Demodulator** (`analog.quadrature_demod_cf`): Recovers the baseband MPX audio spectrum (0 to 120 kHz).
4. **SCA Frequency Translating Filter** (`filter.freq_xlating_fir_filter_fcc`): Shifts the target subcarrier frequency (e.g., 67 kHz or 92 kHz) down to 0 Hz (DC) and applies a tight 5 kHz low-pass filter to reject adjacent MPX energy (such as RDS at 57 kHz).
5. **Narrowband FM Demodulator** (`analog.quadrature_demod_cf`): Demodulates the narrow FM deviation (± 3.5 to ± 7.5 kHz) of the subcarrier into mono audio.
6. **Audio Decimation & De-Emphasis** (`filter.fir_filter_fff` & `filter.iir_filter_ffd`): Decimates audio to 48 kHz and applies a single-pole 75 μ s IIR de-emphasis filter.
7. **Audio Output** (`audio.sink`): Routes 48 kHz PCM audio to system speakers.

Spectrum Visualization Guide (`fm_sca_rx_gui.py`)

The GUI window displays an FFT plot calibrated to show **45 kHz to 105 kHz**:



[45 kHz] ----- (67 kHz Peak) ----- (92 kHz Peak) ----- [105 kHz]

- **Main Audio & Pilot Tone (0–38 kHz):** Shifted off-screen to the left to prevent visual distortion from dominant mono audio.
- **Active FM Subcarrier (67 kHz / 92 kHz):** Appears as a sharp center spike when unmodulated (silent), expanding outward into sidebands (70–75 kHz or 85–98 kHz) during active voice or music passages.
- **Adjusting Gain:** Lower RF gain (-g 30 to -g 35) if broad noise mounds or unmodulated 75 kHz intermod spurs dominate the display.

NOTE:

Viewing a Frequency-Modulated Subcarrier (e.g., 67 kHz or 92 kHz)

When an FM subcarrier (like 67 kHz or 92 kHz) is modulated with audio, its frequency shifts back and forth (± 3.5 kHz to ± 7.5 kHz). On an FFT spectrum analyzer, FM sidebands do not stay static; they expand outward into side lobes during loud audio passages and contract toward the center frequency when silent.

If a station is running a **67 kHz SCA subcarrier** with wide peak deviation (± 7.5 kHz or higher) or heavy compression, the upper sideband energy will peak right around **74–75 kHz** on peak audio swings.

Program Listing: fm_sca_rx_gui.py

```
#!/usr/bin/env python3
```

```
# -*- coding: utf-8 -*-
```

```
# =====
```

```
# FM SCA Subcarrier Decoder (67 kHz / 92 kHz) -- Version 1.0
```

```
# Copyright (C) 2026 Francis M. Columbus
```

```
#
```

```
# System Architecture, DSP Filter Tuning, GNU Radio 3.10 Binding Integration,
```

```
# and DevTest: Francis M. Columbus
```

```
# Code structure developed with AI generation assistance.
```

```
#
```

```
# This program is free software: you can redistribute it and/or modify
```

```
# it under the terms of the GNU General Public License as published by
```

```
# the Free Software Foundation, either version 3 of the License, or
```

```
# (at your option) any later version.
```

```
#
```

```
# This program is distributed in the hope that it will be useful,
```

```
# but WITHOUT ANY WARRANTY; without even the implied warranty of
```

```
# MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
```

```
# GNU General Public License for more details.
```

```
#
```

```
# You should have received a copy of the GNU General Public License
```

```
# along with this program. If not, see <https://www.gnu.org/licenses/>.
```

```
# =====
```



```

import sys
import math
import argparse
from gnuradio import gr, filter, analog, audio, fft, qtgui
from gnuradio.filter import firdes
from PyQt5 import Qt
import sip
import osmosdr

class FmScaReceiver(gr.top_block, Qt.QWidget):
    def __init__(self, station_freq=88.5e6, sca_subcarrier=67000.0, gain=40.0):
        gr.top_block.__init__(self, "FM SCA Subcarrier Monitor")
        Qt.QWidget.__init__(self)

        # Basic Layout Setup
        self.layout = Qt.QVBoxLayout()
        self.setLayout(self.layout)

        # --- Sample Rates & Parameters ---
        self.rf_rate = 2400000    # 2.4 MSps from SDR
        self.audio_rate = 48000   # Standard audio rate
        self.quad_rate = 240000   # Intermediate rate after wideband FM RX

        self.station_freq = station_freq
        self.sca_subcarrier = sca_subcarrier

        # -----

```

1. Hardware Source (RTL-SDR)

```
self.src = osmosdr.source(args="rtl=0")
```

```
self.src.set_sample_rate(self.rf_rate)
```

```
self.src.set_center_freq(self.station_freq, 0)
```

```
self.src.set_gain_mode(False, 0)
```

```
self.src.set_gain(gain, 0)
```

2. Wideband FM Demodulator

```
chan_taps = firdes.low_pass(
```

```
    1.0,          # Gain
```

```
    self.rf_rate, # Input sample rate
```

```
    100000,       # Cutoff frequency
```

```
    20000,       # Transition width
```

```
    fft.window.WIN_HAMMING # GNU Radio 3.10 Window Syntax
```

```
)
```

```
self.rf_filter = filter.freq_xlating_fir_filter_ccc(
```

```
    10,          # Decimation (2.4 MHz -> 240 kHz)
```

```
    chan_taps,
```

```
    0.0,
```

```
    self.rf_rate
```

```
)
```

Quadrature Demodulator for Wideband FM

```

fm_demod_gain = self.quad_rate / (2 * math.pi * 75000)
self.wb_fm_demod = analog.quadrature_demod_cf(fm_demod_gain)

# -----
# 3. Qt GUI Baseband Spectrum Plotter (Focused on 45 kHz - 105 kHz)
# -----
# Center display at 75 kHz with 60 kHz bandwidth
self.qtgui_sink = qtgui.freq_sink_f(
    2048,          # High resolution FFT
    fft.window.WIN_BLACKMAN_HARRIS,
    75000,         # Display center frequency offset
    60000,         # Display bandwidth
    "SCA Subcarrier Spectrum (67 kHz / 92 kHz)",
    1
)
self.qtgui_sink.set_update_time(0.05)
self.qtgui_sink.enable_autoscale(False)
self.qtgui_sink.set_y_axis(-90, -30)

# Embed Qt GUI widget into window
self.pyqtwidget = sip.wrapinstance(self.qtgui_sink.qwidget(), Qt.QWidget)
self.layout.addWidget(self.pyqtwidget)

# -----
# 4. SCA Subcarrier Extraction
# -----
sca_taps = firdes.low_pass(

```

```
1.0,  
self.quad_rate,  
5000,          # 5 kHz cutoff for subcarrier  
1500,          # Transition width  
fft.window.WIN_HAMMING  
)
```

```
self.sca_xlating_filter = filter.freq_xlating_fir_filter_fcc(  
    1,          # Decimation  
    sca_taps,   # Taps  
    self.sca_subcarrier, # Center frequency (67 kHz / 92 kHz)  
    self.quad_rate # Sample rate  
)
```

```
# -----  
# 5. Narrowband FM Demodulation & Audio Output  
# -----
```

```
nb_fm_demod_gain = self.quad_rate / (2 * math.pi * 5000)  
self.nb_fm_demod = analog.quadrature_demod_cf(nb_fm_demod_gain)
```

```
audio_taps = firdes.low_pass(  
    1.0,  
    self.quad_rate,  
    5000,          # Audio cutoff (5 kHz)  
    1000,  
    fft.window.WIN_HAMMING  
)
```

```
self.audio_decimator = filter.fir_filter_fff(5, audio_taps)
```

```
# 75  $\mu$ s De-emphasis Filter via IIR
```

```
tau = 75e-6
```

```
w_c = 1.0 / tau
```

```
alpha = 1.0 / (1.0 + (2.0 * self.audio_rate / w_c))
```

```
b0 = alpha
```

```
b1 = alpha
```

```
a1 = (1.0 - (2.0 * self.audio_rate / w_c)) / (1.0 + (2.0 * self.audio_rate / w_c))
```

```
self.deemph = filter.iir_filter_ffd([b0, b1], [1.0, a1])
```

```
# Audio Sink
```

```
self.audio_sink = audio.sink(self.audio_rate, "", True)
```

```
# Connections
```

```
self.connect(self.src, self.rf_filter)
```

```
self.connect(self.rf_filter, self.wb_fm_demod)
```

```
# Connect Wideband FM output to BOTH Qt Plotter and SCA Filter
```

```
self.connect(self.wb_fm_demod, self.qtgui_sink)
```

```
self.connect(self.wb_fm_demod, self.sca_xlating_filter)
```

```
# Continue DSP path to Audio Sink
```

```
self.connect(self.sca_xlating_filter, self.nb_fm_demod)
```

```
self.connect(self.nb_fm_demod, self.audio_decimator)
```

```
self.connect(self.audio_decimator, self.deemph)
```

```
self.connect(self.deemph, (self.audio_sink, 0))
```

```
def main():
    parser = argparse.ArgumentParser(
        description="GNU Radio FM SCA Receiver with Focused Subcarrier Display"
    )
    parser.add_argument("-f", "--freq", type=float, default=88.5, help="Station frequency in MHz (default: 88.5)")
    parser.add_argument("-s", "--sca", type=float, default=67.0, help="SCA subcarrier frequency in kHz (default: 67.0)")
    parser.add_argument("-g", "--gain", type=float, default=40.0, help="RTL-SDR RF gain in dB (default: 40.0)")

    args = parser.parse_args()

    station_freq = args.freq * 1e6
    sca_subcarrier = args.sca * 1e3

    # Initialize Qt Application Loop
    qapp = Qt.QApplication(sys.argv)

    tb = FmScaReceiver(
        station_freq=station_freq,
        sca_subcarrier=sca_subcarrier,
        gain=args.gain
    )

    tb.show()
    tb.start()
```

```
print(f"\n--- SCA Subcarrier Monitor Running ---")
print(f"Station Frequency: {args.freq:.1f} MHz")
print(f"SCA Subcarrier: {args.sca:.1f} kHz")
print(f"RF Gain: {args.gain} dB")
print("-----")
print("Displaying region: 45 kHz to 105 kHz")
print("Close GUI window to exit.\n")

qapp.exec_()

tb.stop()
tb.wait()

if __name__ == "__main__":
    main()
```

Program Listing: fm_sca_rx.py

```
#!/usr/bin/env python3
```

```
# -*- coding: utf-8 -*-
```

```
# =====
```

```
# FM SCA Subcarrier Decoder (67 kHz / 92 kHz) -- Version 1.0
```

```
# Copyright (C) 2026 Francis M. Columbus
```

```
#
```

```
# System Architecture, DSP Filter Tuning, GNU Radio 3.10 Binding Integration,
```

```
# and DevTest: Francis M. Columbus
```

```
# Code structure developed with AI generation assistance.
```

```
#
```

```
# This program is free software: you can redistribute it and/or modify
```

```
# it under the terms of the GNU General Public License as published by
```

```
# the Free Software Foundation, either version 3 of the License, or
```

```
# (at your option) any later version.
```

```
#
```

```
# This program is distributed in the hope that it will be useful,
```

```
# but WITHOUT ANY WARRANTY; without even the implied warranty of
```

```
# MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
```

```
# GNU General Public License for more details.
```

```
#
```

```
# You should have received a copy of the GNU General Public License
```

```
# along with this program. If not, see <https://www.gnu.org/licenses/>.
```

```
# =====
```



```

import sys
import math
import argparse
from gnuradio import gr, filter, analog, audio, fft
from gnuradio.filter import firdes
import osmosdr

class FmScaReceiver(gr.top_block):
    def __init__(self, station_freq=88.5e6, sca_subcarrier=67000.0, gain=40.0):
        gr.top_block.__init__(self, "FM SCA Receiver")

        # --- Sample Rates & Parameters ---
        self.rf_rate = 2400000    # 2.4 MSps from SDR
        self.audio_rate = 48000   # Standard audio rate
        self.quad_rate = 240000   # Intermediate rate after wideband FM RX

        self.station_freq = station_freq
        self.sca_subcarrier = sca_subcarrier

        # -----
        # 1. Hardware Source (RTL-SDR)
        # -----

        self.src = osmosdr.source(args="rtl=0")
        self.src.set_sample_rate(self.rf_rate)
        self.src.set_center_freq(self.station_freq, 0)
        self.src.set_gain_mode(False, 0)
        self.src.set_gain(gain, 0)

```

```

# -----
# 2. Wideband FM Demodulator
# -----
chan_taps = firdes.low_pass(
    1.0,          # Gain
    self.rf_rate, # Input sample rate
    100000,       # Cutoff frequency (100 kHz)
    20000,        # Transition width
    fft.window.WIN_HAMMING # GNU Radio 3.10 Window Syntax
)

# Decimate RF from 2.4 MHz to 240 kHz (Decim = 10)
self.rf_filter = filter.freq_xlating_fir_filter_ccc(
    10,          # Decimation (Positional)
    chan_taps,   # Taps (Positional)
    0.0,         # Offset (Positional)
    self.rf_rate # Sample rate (Positional)
)

# Quadrature Demodulator for Wideband FM
fm_demod_gain = self.quad_rate / (2 * math.pi * 75000)
self.wb_fm_demod = analog.quadrature_demod_cf(fm_demod_gain)

# -----
# 3. SCA Subcarrier Extraction
# -----

```

```

sca_taps = firdes.low_pass(
    1.0,          # Gain
    self.quad_rate,  # Input rate
    7500,         # Cutoff frequency (7.5 kHz for SCA)
    2500,         # Transition width
    fft.window.WIN_HAMMING
)

# Positional C++ bindings (pybind11)
self.sca_xlating_filter = filter.freq_xlating_fir_filter_fcc(
    1,           # Decimation
    sca_taps,    # Taps
    self.sca_subcarrier, # Center frequency (e.g., 67000 Hz)
    self.quad_rate # Sample rate
)

```

```

# -----

```

```

# 4. Narrowband FM Demodulation & Audio Output

```

```

# -----

```

```

nb_fm_demod_gain = self.quad_rate / (2 * math.pi * 5000)
self.nb_fm_demod = analog.quadrature_demod_cf(nb_fm_demod_gain)

```

```

# Resample to 48 kHz Audio Rate

```

```

audio_taps = firdes.low_pass(
    1.0,
    self.quad_rate,
    5000,          # Audio cutoff (5 kHz)

```

```
    1000,  
    fft.window.WIN_HAMMING  
)  
self.audio_decimator = filter.fir_filter_fff(5, audio_taps)
```

```
# 75  $\mu$ s De-emphasis Filter via IIR
```

```
tau = 75e-6
```

```
w_c = 1.0 / tau
```

```
alpha = 1.0 / (1.0 + (2.0 * self.audio_rate / w_c))
```

```
b0 = alpha
```

```
b1 = alpha
```

```
a1 = (1.0 - (2.0 * self.audio_rate / w_c)) / (1.0 + (2.0 * self.audio_rate / w_c))
```

```
self.deemph = filter.iir_filter_ffd([b0, b1], [1.0, a1])
```

```
# Audio Sink
```

```
self.audio_sink = audio.sink(self.audio_rate, "", True)
```

```
# Connections
```

```
self.connect(self.src, self.rf_filter)
```

```
self.connect(self.rf_filter, self.wb_fm_demod)
```

```
self.connect(self.wb_fm_demod, self.sca_xlating_filter)
```

```
self.connect(self.sca_xlating_filter, self.nb_fm_demod)
```

```
self.connect(self.nb_fm_demod, self.audio_decimator)
```

```
self.connect(self.audio_decimator, self.deemph)
```

```
self.connect(self.deemph, (self.audio_sink, 0))
```

```
def main():
    parser = argparse.ArgumentParser(
        description="GNU Radio FM SCA Subcarrier Receiver"
    )

    parser.add_argument(
        "-f", "--freq",
        type=float,
        default=88.5,
        help="Station frequency in MHz (default: 88.5)"
    )
    parser.add_argument(
        "-s", "--sca",
        type=float,
        default=67.0,
        help="SCA subcarrier frequency in kHz (default: 67.0)"
    )
    parser.add_argument(
        "-g", "--gain",
        type=float,
        default=40.0,
        help="RTL-SDR RF gain in dB (default: 40.0)"
    )

    args = parser.parse_args()

    station_freq = args.freq * 1e6
```

```

sca_subcarrier = args.sca * 1e3

tb = FmScaReceiver(
    station_freq=station_freq,
    sca_subcarrier=sca_subcarrier,
    gain=args.gain
)

print(f"\n--- SCA Receiver Running ---")
print(f"Station Frequency: {args.freq:.1f} MHz")
print(f"SCA Subcarrier: {args.sca:.1f} kHz")
print(f"RF Gain: {args.gain} dB")
print("-----")
print("Press Ctrl+C or Enter to stop.\n")

tb.start()
try:
    input()
except KeyboardInterrupt:
    pass
tb.stop()
tb.wait()

if __name__ == "__main__":
    main()

```